Operating LLM-Generated Data Wrappers in Production: A 149-Day Field Study of What Breaks in a Smart-City Intake Layer

Anonymous Author(s)

***Abstract*—The Observatory is a smart-city observatory whose intake layer is built from LLM-generated data wrappers: a non-technical user registers a source through a form, and the platform writes, runs, and repairs the extraction code itself, mapping REST APIs, spreadsheets, and CSV files to a normalized (timestamp, value) series. We report 149 days in production, 396 wrappers and 164,322 measurements. The binding faults turn out to be environmental, not bad code: almost all 69 terminal errors are watchdog kills of long-cadence pollers or operational faults (spent quota, provider overload, a mid-generation restart), and the one error-status wrapper whose generated code went unrepaired had its repair blocked by that same quota. Cheap generation also decouples attempt from yield, only 284 of 396 wrappers ever produced data, most of the silent remainder deleted or never run rather than failed extractions; and crash-triggered repair can mask faults as easily as fix them, so we downscope "recovery" to restored execution. The MAPE-K loop and its LLM actuator are an organizing lens and the repair mechanism is established program repair; what is new is production evidence of where such a system breaks in a civic observatory.**

***Index Terms*—production deployment study, LLM-generated data wrappers, LLM code generation and repair, format-adaptive data ingestion, smart-city observatory, autonomic computing, self-healing systems**

I. Introduction

Running a city on evidence means tracking many numbers at once, and those numbers neither sit in one place nor arrive in one form. Environmental readings come off sensor APIs, tourism figures from boards, energy and mobility counts from utilities and transport operators, each with its own way in, a REST endpoint here, an Excel or CSV export there, and its own field names, date formats, and language. None of it stands still: an indicator earns its place only while it stays current, so every feed must be pulled again and again, and every upstream change of format or endpoint absorbed without breaking the series already built. What an observatory confronts is a standing integration problem over many heterogeneous sources, not a task finished once.

The usual way to take a source in is to write an extraction wrapper for it and then keep that wrapper alive as the source shifts underneath. This cost, paid once per source to build and then indefinitely to maintain, is why mediator-style integration has never reached past a small set of feeds [1]–[3], and it sets the ceiling on coverage: an observatory ends up showing whichever feeds someone found time to wire and to nurse, rather than the city.

We call an intake layer *format-adaptive* when onboarding a feed costs no per-source extraction code: a domain user registers the source and its metadata through a form, and the platform writes, runs, and maintains the wrapper from there. The adaptivity is bounded, and we are explicit about it: the platform adapts to a source's format and transport within the structured tabular family, REST/JSON APIs, spreadsheets, and delimited files, all projected onto one univariate (timestamp, value) series. It is not adaptivity to data modality or schema. This paper reports the Observatory (anonymized for review), a smart-city observatory running for a European region, where format-adaptive intake is obtained by making each wrapper an *autonomic* element in the sense of Kephart and Chess [4]: a component that manages itself through a closed loop over its own lifecycle. The actuator here is a large language model (LLM) rather than a parameter tuner: it writes the component's code and rewrites it after a failure. We use this only to describe the setting; the loop itself is established LLM program repair, not our contribution (Section II).

We organize the wrapper's three self-* behaviours as the monitor–analyze–plan–execute loop over shared knowledge (MAPE-K) of the autonomic-computing blueprint [5], [6]. At registration, *self-generation* (Section IV-B) plans the artifact: the LLM completes a fixed template from the indicator's metadata and a data sample, interrogating a live API through a controlled fetch tool before it writes anything. *Self-deployment* (Section IV-C) is the execute step that runs unattended: each wrapper is an isolated process the platform starts, watches, check-points, and resumes from per-wrapper watermarks after any restart. *Self-healing* (Section IV-D) is the loop closing on a live artifact: a crash is detected, the termination is classified, the model re-plans from the captured trace, and the corrected wrapper goes back out.

The MAPE-K loop is a *lens* we use to organize the system and to name the boundary of what the loop can fix; it is not a novelty claim. The contributions are the architecture and its one narrow contract (Section III) and, mainly, three empirical findings from the 149-day run. First, the *binding faults are environmental, not bad code*: almost all 69 terminal errors are watchdog kills of long-cadence pollers or opera-

tional faults (spent quota, provider overload, a mid-generation restart), and the only error-status wrapper whose generated code went unrepaired was itself blocked by a spent quota (Section V-C). Second, *cheap generation decouples attempt from yield*: only 284 of 396 wrappers ever produced data, shifting the bottleneck from authoring to triage of the silent tail (Section V-A). Third, *crash-triggered repair can mask faults rather than fix them*: a case repair can recover by swallowing an upstream error instead of extracting the intended data, so restored execution is not validated data (Sections V-B and V-D). We treat the in/out fault partition as a useful framing of the first finding, not as a discovery in itself, since the partition is drawn by definition.[1]

II. Background and Related Work

A. Smart-city and tourism observatories

The smart-tourism literature has long argued for destination-scale data infrastructure [7], [8] and has codified the indicators worth tracking through schemes such as ETIS [9], while field reports describe patchy uptake [10] and dashboards assembled mostly from warehouses and surveys [11]. A recurring finding in evaluations of smart-destination initiatives is that the binding constraint on sustained monitoring is getting the data in at all [12], [13]. These accounts say little about how intake is actually built. Our claim is that the cost of taking in and maintaining a feed, more than the analytics on top, is what bounds how many indicators an observatory can sustain, and we report an intake layer that drives that cost down, from production. Cheap intake widens what an observatory can attempt; not every attempt yields a live indicator, as our yield numbers later show. The deployment happens to be a tourism observatory, but nothing in the design ties it to tourism: it needs only structured tabular sources that map onto a shared time-series target.

B. Autonomic computing and self-* systems

Autonomic computing proposed self-managing components with closed control loops as the response to operational complexity [4], structured as a monitor–analyze–plan–execute cycle around a shared knowledge base (MAPE-K) [5], and later surveys catalogue its self-* property families [6]. The Observatory applies that blueprint to data wrappers, but swaps the actuator: instead of tuning a fixed component's parameters, the actuator is an LLM that regenerates the component's source code, the move a recent roadmap on generative AI for self-adaptive systems surveys [14]. We use this correspondence to organize the exposition, not as a novelty claim.

[1]For review, the platform's services are mirrored anonymously on Anonymous GitHub: the orchestrator (https://anonymous.4open.science/r/tsd-orchestrator-84B0), the resource service (https://anonymous.4open.science/r/tsd-resource-service-D9C4), the indicator service (https://anonymous.4open.science/r/tsd-indicator-service-7A1F), the data collector (https://anonymous.4open.science/r/tsd-data-collector-3B82), the user service (https://anonymous.4open.science/r/tsd-user-service-C641), the user interface (https://anonymous.4open.science/r/tsd-user-interface-5E9A), and the knowledge base (https://anonymous.4open.science/r/tsd-knowledge-base-2F47). The reported figures are computed from a production-state snapshot by an analysis script we will release with the camera-ready.

C. LLMs for code generation, repair, and extraction

Language models now write runnable programs from prose [15], and automated repair has moved toward LLMs as well [16]. Several systems close a feedback loop around the generated code, whether through self-critique [17], episodic reflection [18], or conditioning on execution results [19], though the benefit of self-repair is inconsistent [20]. Repair has also run in production: SapFix and Getafix ship automated fixes for crashes and bugs at industrial scale [21], [22], and agentic LLM repair drives the generate–run–observe–revise loop autonomously [23]. On the data side, Evaporate established that synthesizing extraction code is cheaper at scale than calling a model per record [24], AutoScraper builds reusable scrapers with an agent [25], agentic systems are now benchmarked on enterprise data workflows [26], and both prompted [27] and fine-tuned [28] models handle textbook preprocessing steps. Detecting when such a pipeline silently degrades is itself a line of work [29], and our masking finding is one motivation for it. **We make no novelty claim for the loop itself**: the repair is established LLM program repair, the generation is Evaporate-style synthesis, and the probing is ordinary tool use. What these benchmark systems do not provide, and what we contribute, is a measured population of *field* faults from a live deployment, from which we read where the loop's repair helps and where it cannot. The production-crash trigger, whole-artifact repair, and fixed univariate contract are how our setting differs, not advances we claim over those systems.

D. Wrapper induction, maintenance, and ETL automation

Early wrapper induction derived extraction rules from labelled examples [2], later systems learned them without supervision from structural regularities [30], and the tooling has been surveyed at length [31], [32]. Keeping a wrapper working as its source drifts, by spotting the break and re-deriving the rules, was identified as a distinct problem from the start [3], [33]. ETL pipelines, meanwhile, have mostly been designed by hand rather than generated [34], [35], with automation reaching adjacent pieces such as schema matching [36], large-scale curation [37], and wrangling [38]. The Observatory folds both the initial derivation and the later re-derivation into LLM code generation, which reaches past template-structured pages to the REST APIs, spreadsheets, and delimited files an observatory ingests, and it targets exactly the slice an indicator platform needs: pulling one series from one source into one normalized schema. A separate line of integration research makes data source-agnostic for the *consumer* by deferring or learning structure, in pay-as-you-go dataspaces [39], polystores over diverse engines [40], and schema-on-read data lakes; we work the orthogonal, producer side, building and repairing each source's wrapper through an autonomic LLM loop. The heterogeneity we span is one of format and transport; the target stays a single univariate series.

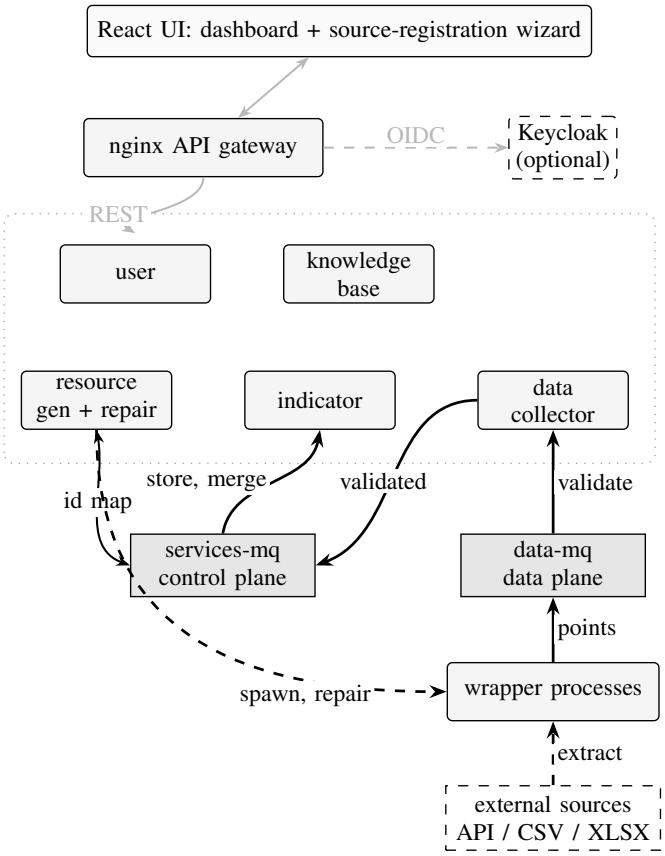


Fig. 1. Whole-system architecture. The React front end reaches the Python/-FastAPI services through the nginx gateway (the REST serving plane, thin grey arrows); ingestion runs asynchronously across two RabbitMQ brokers, a data plane and a control plane (solid black arrows); and wrapper processes that the resource service spawns and repairs read the external sources (dashed arrows). Each service keeps its own datastore (MongoDB, with Redis at the indicator service and data collector), and an optional Keycloak provider handles single sign-on.

III. Architecture and the Fixed Time-Series Contract

The Observatory runs as a set of containerized services, user, knowledge-base, data-collector, resource, and indicator (all Python and FastAPI), reached through an nginx gateway and fronted by a React dashboard. Control traffic and data traffic ride separate RabbitMQ brokers, and the resource and indicator services each keep their own datastore, with an optional Keycloak provider for single sign-on. Figure 1 lays out these components and the two planes that join them: a REST serving plane through the gateway, and an asynchronous ingestion plane across the brokers along which the wrapper processes emit their points.

Format-adaptive intake rests on one deliberately thin interface. Whatever its format, a wrapper emits point batches $\{(x, y)\}$ in which x is an ISO 8601 timestamp and y a floating-point value; the code above that interface is generated per source, and everything below it is shared, fixed plumbing. The thin contract is also the boundary: a source is in scope precisely when it can be projected onto a single $(\text{timestamp}, \text{value})$ series. Out of scope by construction are unstructured or scanned content (PDFs, images, free text), push and streaming transports (Kafka, MQTT, websockets), multivariate, high-frequency, geospatial, and graph data, and anything that needs a join or entity resolution across feeds; the unhealable single-column CSV of Section V-D is this boundary made concrete. A user registers a source through a web wizard (a file upload, or an API endpoint and its credentials) with the indicator's metadata; the resource service then owns that wrapper for life, and downstream the collector validates each batch against the contract before the indicator service stores, merges, and serves the series. Source type is one argument to a single generation routine, not a code path of its own, so the same routine handles all three formats.

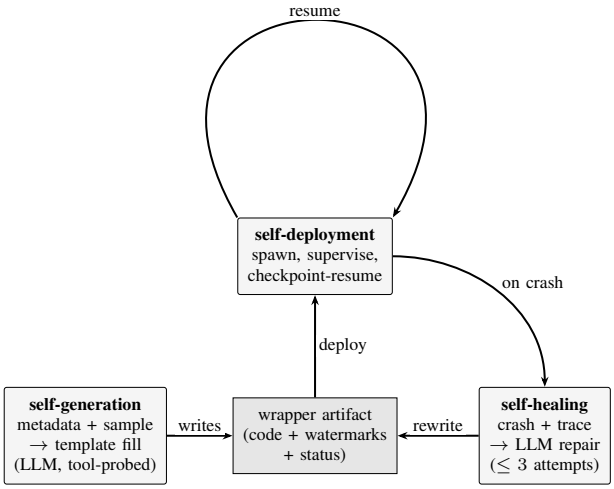


Fig. 2. One manager over one wrapper artifact, with the three self-* behaviours arranged clockwise around it. Self-generation writes the artifact, self-deployment runs and resumes it, and self-healing rewrites it after a crash for redeployment; the actuator throughout is an LLM paired with a process supervisor.

IV. Self-* Wrappers as an Autonomic Manager

Each wrapper is the element a single autonomic manager looks after. That element is the generated artifact plus the state that travels with it: the wrapper's code, its execution watermarks, its retry counter, and its status. The platform turns the MAPE-K loop [5] on that element, watching process state, classifying how a process ended, planning a fresh or corrected artifact, and carrying the plan out by deploying it, all against a shared store of knowledge. We use this loop only as an organizing lens; its value here is that it names *which faults the manager has authority over* (Section IV-E). The three self-* loops in Figure 2 are what it yields. In every one of them the actuator is the same pairing: an LLM that produces and revises code, and a process supervisor that puts it into service.

A. Shared knowledge

What binds the loops into one manager, rather than three disconnected mechanisms, is the knowledge they share. Some

of it is fixed. The wrapper template encodes the Section III contract as code, and a standing list of fourteen hard rules captures lessons from earlier failures, among them: leave the historical-backfill driver alone, invent no data, treat date ranges as inclusive, keep timezone handling uniform, and read non-English dates such as localized month names. Together these constrain what the planner is allowed to emit. The rest is per-wrapper and changes at runtime: a pair of watermarks (the earliest and latest timestamps sent) with a phase, a retry counter capped at three, a timestamped execution log, and the stored wrapper document carrying its metadata, source configuration, current code, and status. The generation loop writes the first version of the code, the deployment loop reads and advances the watermarks, and the healing loop reads the code and the log and writes a successor. The rule list is the part that grows: each diagnosed failure can add a constraint that every later plan must respect.

B. Self-generation: planning a new artifact

The actuator is Google's Gemini Flash, reached through the `google-genai` SDK at its default decoding settings (no temperature or top-p tuning) and pinned to the floating `gemini-flash-latest` alias. It does not write code freely but fills a fixed wrapper class (Figure 3): only extraction (`fetch_external_data`) and, for APIs, a windowed fetch and a polling period are open to it, while the backfill driver, transport, batching, and run loop are library code the prompt tells it not to touch. Jinja2 assembles the prompt from the indicator metadata, the source configuration, and a brief data sample, the opening 20 lines of a CSV, 15 rows per workbook sheet with the sheet names, or a captured API reply, alongside the required `{x,y}` output shape, the template, source-type instructions, and the rule list. For an API, the same call gives the model a sandboxed `fetch_url` tool returning the first 2500 characters of a live reply, which it may call up to fifteen times to learn the response's nesting, field names, and date filters rather than guess them from the URL. Request authentication is built into the headers by the service from the declared scheme, so the model never handles it. A static check gates deployment, parsing the file and looking for placeholders left unfilled; a first failure earns one corrective prompt, a second sends the wrapper to `error`. Nothing is executed as a test beforehand, so any runtime fault is the healing loop's problem.

C. Self-deployment: execute without an operator

Deployment is ordinary process supervision, and we claim nothing new for it; we record it only because it is what lets the other two loops run unattended. The resource service runs each wrapper as its own operating-system process under a concurrency semaphore (default 100) and a per-run watchdog (default 3600 s). A file wrapper runs once; an API wrapper back-fills history in size-adapting batches and then polls at the period its periodicity implies, with no external scheduler. A monitor reads process state every 30 s. Each wrapper keeps a low and a high watermark (its earliest and latest emitted timestamps);

```
class Wrapper:
    # fixed: transport, backfill, run loop (do not edit)
    async def fetch_historical_data(self):
        return await self.backfill(
            self.fetch_date_range, self.send_to_queue)

    # variable: the model fills only these
    async def fetch_external_data(self):
        """return [{'x': <ISO-8601>, 'y': <float>}, ...]"""
        ... # source-specific extraction
    async def fetch_date_range(self, start, end):
        ... # API only: windowed fetch
    def get_interval_seconds(self):
        ... # API only: poll period
```

Fig. 3. The wrapper contract (shortened). Only the open methods are the model's to fill; backfill, transport, and scheduling are fixed. Holding generation to the extraction logic is what reduces source type to an argument instead of a codebase.

after a restart it resumes from the high mark rather than re-fetching, so a relaunch neither duplicates nor skips points, the same check-point-and-resume idiom as streaming consumers (Kafka offsets, Flink checkpoints). Generation produces a process to oversee; healing produces a replacement to relaunch and resume.

D. Self-healing: the full loop on a live artifact

Healing is where every MAPE-K stage shows up at once. The 30 s monitor reads each process's exit status (*monitor*). An abnormal exit, in practice always code 1, is separated from an idle watchdog kill and from a provider error during generation, and only the first becomes a repair (*analyze*). The plan is an LLM call carrying the full crashed source, its captured stderr, and the final twenty lines of stdout, with the instruction to return a complete corrected file that preserves the class and addresses that particular traceback (*plan*); for an API wrapper it may re-interrogate the endpoint through the same `fetch_url` tool, up to ten times. The returned file goes through the same static check as a first generation, replaces the old file, bumps the retry counter, and the process is relaunched and resumed from its watermarks (*execute*). The manager grants at most three repairs to any wrapper and leaves the fourth-time loser in `error` for a person. Two things come from treating the artifact, not a diff, as the unit of repair. The trigger is a live exit code that may arrive weeks after generation, so the feedback carries the source's real behaviour rather than a test author's guess; and because the plan re-emits the whole wrapper, a repair can re-shape the extraction end to end instead of touching one statement [19], [20].

E. Scope of control

Format-adaptivity then comes out of the manager rather than from any separate component. One knowledge base and one set of loops cover API, spreadsheet, and CSV feeds because source type is a parameter the template takes, so a feed in one of those families is taken in simply by generating, deploying, and repairing its wrapper with no per-source code. The same view marks the limit of the manager's reach. We draw the boundary operationally, before looking at the counts: a fault is *inside* the manager when a change to the wrapper's code can

fix it (broken extraction logic, a source's formatting quirks), and *outside* when the fix lies in the environment the code cannot rewrite (a spent model quota, a stale credential, a watchdog set shorter than a source's polling period). Section V-C then asks how the production faults fall relative to that line. The boundary is a design choice, not a law: the watchdog and polling period are themselves values the generator sets, so a poll-interval mismatch is environmental only because we left interval-setting outside the loop, and it could be brought inside.

V. Production Deployment and Metrics

The figures here come from a 149-day run (January 13, 2026 to June 11, 2026) captured as one complete state snapshot: the wrapper collection (396 documents, generated code and execution logs included), the per-wrapper logs, the resource and indicator stores, the service logs, and container statistics; every number below is computed from that snapshot. At the snapshot the deployment spanned 17 containers and about 2,082 MiB of resident memory while idle, dropping to roughly 1.5 GiB once the optional single-sign-on container is set aside; the resource service, host to every wrapper process, was the heaviest at 682 MiB.

A. Generation and coverage

The manager produced 396 wrappers, 188 for spreadsheet sources, 184 for APIs, and 24 for CSV, distributed over the three domains (environment 203, economy 139, sociocultural 54); between them they collected 164,322 points (53,530 from spreadsheets, 79,411 from APIs, 31,381 from CSV). Volume is long-tailed: the median wrapper contributed 27.5 points and the 90th percentile 967, with 284 of 396 sending at least one. The time from a generation request to the first running process had a median of 26.8 s and a 90th percentile of 109.8 s over 331 wrappers, a figure that folds in queueing, sampling, the model call, and the static check. Further down, 118 of the 179 active indicators are tied to a source and 111 carry data through the whole pipeline; the busiest live series is an hourly air-quality feed at 20,069 points, and backfill reaches the year 2000 for the 11 feeds whose lower watermark was retained. Cheap generation comes with a long silent tail: 112 wrappers delivered no data, 69 with a since-deleted resource, 20 that never ran (quota or watchdog), 22 that completed yet found nothing usable, and 1 still executing, so scaling coverage shifts the cost from authoring wrappers to triaging this tail. Outcomes by source type appear in Table I, and the status machine with production counts in Figure 4.

B. Self-healing effectiveness

43 wrappers, 10.9% of the total and split 26 spreadsheet, 14 API, and 3 CSV, threw an uncaught exception while running and were taken up by the repair loop, which shipped 50 redeployments; counted per wrapper, the repair depth was 37 at a single attempt, 5 at two, and 1 at three. "Recovery" can be read three ways, and they do not agree, so we keep them apart. Measured by recurrence, the same crash came back after the last repair for exactly one of the 43, so 42 stayed crash-free. Measured by final status, 39 of 43 (90.7%) closed in a non-error state; that reading is by final status alone, so it still counts the one wrapper that burned all three attempts and never stabilised, which shows a non-error status only because its resource was deleted afterwards and which we do not score as a win. Measured by output, 32 resumed sending data after their last repair, 20,351 points, about an eighth of everything collected, though a single test feed accounts for 17,466 of them. The repairs themselves were fast and untouched by a person: a median 33.4 s from crash to redeployment (quartiles 24.2 and 50.8 s, 90th percentile 60.5 s, longest 80.4 s over the 50 events), up to 30 s of which is the detection cycle. The typical crash arrived 16.8 s into a run, in the fetch-and-parse stage, and none occurred while a poller waited idle.

The run also offers a pre-automation reference period. For the first 65 days no repair loop was active (it first appears in the logs on 2026-03-19), and over that stretch 36 crashes of this same exit class were simply left to die; we read the comparison descriptively, since the early period differs in source mix and load and is not a matched control. Two features of the timeline qualify the healing counts. Thirty-one of the 50 repair events land on a single day, when a credential fix set off a redeploy that restarted every wrapper together, so the events are correlated rather than 50 independent failures. The latency is unaffected. The headline 33.4 s is over all 50 repair events; restricted to each wrapper's first repair, the 17 that first crashed on other days have a median 34.0 s (range 15.5–

TABLE I

Per-source-type outcomes over the 149-day run. *Recovered* counts wrappers that went through the repair loop and closed in a *non-error final status* (39 of 43, one of them the retries-exhausted wrapper that reached non-error only through later resource deletion); this is a stricter measure than the 42 of 43 whose crash simply never recurred (Table II). One set of loops handles every source type.

Type	n	Completed	Crashed	Recovered	Points
XLSX	188	104 (55.3%)	26	26	53,530
API	184	52 (28.3%)[a]	14	10	79,411
CSV	24	14 (58.3%)	3	3	31,381
All	396	170 (42.9%)	43	39	164,322

[a]Continuous API pollers seldom finish on their own; 54 reached `error` when the watchdog stopped them mid-idle (Section V-C), which is not a code fault.

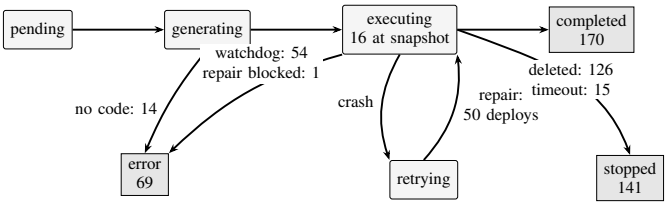


Fig. 4. Status machine, annotated with production counts ($n = 396$). The crash-and-repair branch touched 43 wrappers across 50 redeployments; most `error` outcomes are watchdog kills of long-cadence API pollers rather than faulty code (Section V-C).

TABLE II
WHY THE 69 ERROR-STATUS WRAPPERS ENDED THERE, AND HOW THE REPAIR LOOP FARED OVER THE 43 CRASHED WRAPPERS AND THEIR 50 AUTOMATIC REPAIRS.

(a) Root causes of `error` status ($n = 69$)		
Stage	Root cause	n
Generation	Model daily free-tier quota (HTTP 429)	9
Generation	Model per-request rate limit (HTTP 429)	1
Generation	Model provider overloaded (HTTP 503)	1
Generation	Service restart during generation	3
Runtime	Execution watchdog (3600 s), API pollers[a]	54
Runtime	Crash; repair blocked by LLM quota (429)	1
	Total	69
(b) Self-healing outcomes (43 wrappers, all crashes exit 1)		
Healed after 1 repair (crash never recurred)		37
Healed after 2 repairs		5
Still crashing after 3 repairs (retries exhausted)		1
Final status: completed / stopped / executing		28 / 8 / 3
Final status: error (later watchdog timeout)[b]		4
Crash-free after repair (recurrence)		42/43
Median crash-to-redeploy repair latency		33.4 s

[a]stderr held no traceback (53 of 54 blank, one harmless warning); the kills happened during idle waits between polls, the one-hour watchdog being tighter than these feeds' daily or monthly periods. By then 48 of the 54 had shipped 52,372 points. [b]Each of the four was repaired and never crashed again; the later `error` comes from the watchdog limit, not the repaired code.

80.4) against 33.2 s for the 26-wrapper batch, so the figure is not an artifact of the mass relaunch. The recovery counts hold the same way: 15 of those 17 independent crashes ended in a non-error state, an 88% rate in line with the pooled 39 of 43 (90.7%). Because each relaunch overwrites stderr, a triggering trace survives only where the last run also crashed; for recovered wrappers we read the cause from the fixes their code carries, so these counts describe the model's *repairs*, not independently observed causes (a defect-referencing comment or guard appears in 7 of the 43 crash wrappers and none of the 353 clean ones).

C. Faults inside and outside the manager

The result in this section is the paper's central one: the binding faults are environmental, not bad code. Table II sorts the 69 wrappers that ended in `error` by cause, and few of them indict the generated code. Fourteen never reached the running stage: nine ran into the model's daily free-tier quota, one a per-request rate limit, one a provider overload, and three more were cut off mid-generation when a service restarted. Another, larger set of 54 are API pollers the 3600 s watchdog stopped between polling cycles, 53 of them leaving no traceback; here the watchdog is just shorter than the source's daily or monthly rhythm, a setting mismatched to the feed rather than broken code, and 48 of these had already delivered 52,372 points before being cut short. One wrapper crashed and could not be mended because the repair call itself struck the quota ceiling, and that is the lone error-status case where real generated code went unrepaired. The lesson is about the workload, not just the loop: in a production indicator platform the faults that bind are environmental, not defects in the generated code. The specific 3600 s watchdog and free-tier quota are this deployment's instantiation; what we expect to transfer is the category, that once generation is cheap, code defects grow rare relative to the operational envelope (quota, credentials, cadence-to-timeout matching). The split also makes the scope of control of Section IV-E concrete: code defects and formatting quirks come off the operator's plate, while spent quota, bad credentials, and cadence-mismatched limits stay outside the managed element and stay manual. The manager removes the per-source *code*, not every per-source decision.

D. Repair case studies

Four wrappers put faces on the partition. Two of the four "heal" by absorbing an upstream error rather than extracting the intended data, a third makes a benign clean, and the last is out of scope. A yearly urban-waste API wrapper crashed 82 s into its backfill on a timezone-aware-versus-naive comparison; the repair normalizes both bounds and downgrades an empty fetch to a log line (back in 34.7 s, 596 points). An air-quality wrapper whose configured URL already pinned a date window had a second one appended and got an HTTP 400; the repair strips the duplicate parameters and swallows any residual 400 (51.3 s, 395 points, still streaming). A cultural-heritage workbook repeated its year header (`"Ano"`) among the values, so the wrapper died on `int("Ano")`; the repair coerces to numeric and drops non-numeric rows, a fix three sibling wrappers received the same night (61.6 s). The one out-of-scope case is a search-interest export that arrived as a single column: three repairs (27.0/33.0/51.8 s) tried delimiter and bad-line tricks but could not invent a $(\text{time}, \text{value})$ pair, while a two-column sibling of the same export healed on the first try. The manager mends code; it cannot supply data the source withholds.

E. Pipeline integrity and throughput

Point counts thin out down the pipeline: of the 164,322 points the wrappers emit, 148,131 reach the resource store as raw segments and 106,442 survive consolidation there, 100,550 land in the indicator store, and chronological merging leaves series of 84,431. The losses trace to deleted resources (198 of 396, 126 of which left their wrapper `stopped`), to regeneration wiping earlier data, to points landing for resources with no indicator attached, and to timestamp-level deduplication at merge time. Where we could check integrity, it held: across 30 days the collector rejected nothing on schema grounds and raised only 18 duplicate-timestamp warnings, each settled by averaging; over an 8.3-hour control-plane window the resource service emitted 153 messages and the indicator service took in exactly 153, none lost in the broker. Resumption proved itself in the field: a container rebuild a few hours before the snapshot sent all 16 running wrappers through a restart, 15 of them continuing from stored watermarks while

the one missing a high watermark re-read its history from scratch.

VI. Discussion

Seen as an autonomic manager, the deployment explains both its successes and its ceiling. Source-specific code stays out of human hands because generation is fenced in by a fixed contract, because the plan is grounded in what the source actually returns instead of what it is assumed to return, and because repair fires on a genuine production trace. The ceiling is precisely the faults that sit outside the managed element: the single error tied to unrepaired code was waiting on a quota, and the single crash the loop could not mend was over data that was not there.

For a city scaling an indicator observatory the three findings carry into operations. Since the binding faults are environmental, coverage grows by governing quotas, credentials, and polling cadences rather than by chasing a better code generator: cheap generation moves the bottleneck off authoring and onto operations. Since attempting a feed is nearly free, an observatory can take on the long tail of feeds it once skipped across all its domains. And since repair can mask data loss, a dashboard may show a healthy wrapper that has silently stopped delivering, a real trust hazard when the series inform evidence-based policy, so value-level monitoring becomes a first-class need rather than an afterthought.

The evidence has limits worth stating plainly. It is observational, drawn from one deployment, one model family, and one region, with no side-by-side control; the pre-automation period is a reference rather than a matched arm, and the crash events are correlated, not independent. We have no hand-written wrappers for the same sources to measure against, because none were ever built. We score a wrapper as recovered when it returns to a non-error state, which (as the deepest-retry case shows) need not mean it ran cleanly and says nothing about whether the values it extracts are right, since the collector inspects each batch's shape and never the underlying truth. We could not close this gap from the snapshot: it retains no copy of the original sources, and the downstream stores are confounded by resource deletion, regeneration, and merge-time deduplication, so an extracted value cannot be checked against ground truth. A best-effort look at the healed wrappers' final stored series is consistent with this caution rather than reassuring against it: many are sparse or single-valued, and two of the four case-study repairs work by *swallowing* an upstream error (downgrading an empty fetch to a log line, absorbing a residual HTTP 400), which is exactly the move that can mask data loss behind a clean exit; a third drops non-numeric rows to clear a repeated header, a benign clean here but the same blunt operation. We therefore down-scope "self-healing" to restored *execution* and treat value-correctness as open. Relatedly, for the healed wrappers the cause is read from the repair because the relaunch destroys the trace, so the fault analysis characterizes the model's fixes more than independently observed failures. The model is pinned only through the floating `gemini-flash-latest` alias, so the exact numbers are not reproducible against a fixed model and may reflect weight changes mid-run; the generation latency includes queueing and so overstates model time; the memory figure is an idle reading; and watermark coverage spans are computable for only the 11 wrappers whose low mark was kept. We do not ablate the design choices the system rests on, tool-assisted probing, the template constraint, and whole-file repair, so their value is argued from design rather than measured; and because the source mix is user-selected and the fourteen hard rules accreted over the run, the low rate of code defects is partly co-produced by that human curation, a maturation effect we cannot separate out. The four repair case studies are illustrative, chosen to exercise the scope boundary rather than sampled at random, so they bound where masking can occur, not how often. Whether the pattern carries beyond 396 wrappers and one region is argued from its design, not demonstrated.

A production smart-city platform also raises governance questions we touch only briefly. The indicators are aggregate public statistics with no personal or record-level data, so the deployment poses no direct privacy or GDPR concern, but two operational risks are real. The model quota is a single point of failure, already behind 11 of the 69 terminal errors. And running model-written code as live processes is contained here only by per-process isolation, a concurrency semaphore, and the execution watchdog, not by a stronger sandbox; the generation prompt also ingests a sample of the live source (CSV lines or an API reply), an untrusted-input path that a prompt-injection attack could in principle exploit, and one we did not harden.

VII. Conclusion

We operated LLM-generated extraction wrappers as live infrastructure for 149 days: 396 wrappers, 164,322 points, and of the 43 that crashed, all but one stayed crash-free after at most three unattended repairs (a stricter read by final status leaves 39 of 43 non-error and 32 still delivering data), at a median 33.4 s that holds with the one correlated mass-relaunch set aside. The result we stand behind is empirical: in production the faults that bind are environmental, not defects in the generated code, and the repair loop governs exactly the code the LLM owns. The in/out partition is the framing that organizes that finding, not a discovery in itself. What the loop could not absorb was environmental, which sets the agenda: checking extracted values for meaning, and folding quota and credential faults into the manager as state it owns rather than leaving them on an operator's desk.

References

[1] G. Wiederhold, "Mediators in the architecture of future information systems," *Computer*, vol. 25, no. 3, pp. 38–49, 1992.

[2] N. Kushmerick, "Wrapper induction: Efficiency and expressiveness," *Artificial Intelligence*, vol. 118, no. 1–2, pp. 15–68, 2000.

[3] K. Lerman, S. N. Minton, and C. A. Knoblock, "Wrapper maintenance: A machine learning approach," *Journal of Artificial Intelligence Research*, vol. 18, pp. 149–181, 2003.

[4] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[5] IBM Corporation, "An architectural blueprint for autonomic computing," IBM, White Paper, Jun. 2006.

[6] M. C. Huebscher and J. A. McCann, "A survey of autonomic computing—degrees, models, and applications," *ACM Computing Surveys*, vol. 40, no. 3, pp. 7:1–7:28, 2008.

[7] U. Gretzel, M. Sigala, Z. Xiang, and C. Koo, "Smart tourism: foundations and developments," *Electronic Markets*, vol. 25, no. 3, pp. 179–188, 2015.

[8] D. Buhalis and A. Amaranggana, "Smart tourism destinations," in *Information and Communication Technologies in Tourism 2014*, Z. Xiang and I. Tussyadiah, Eds. Cham: Springer International Publishing, 2013, pp. 553–564.

[9] European Commission, Directorate-General for Internal Market, Industry, Entrepreneurship and SMEs, "The European Tourism Indicator System: ETIS toolkit for sustainable destination management," Publications Office of the European Union, Luxembourg, Tech. Rep., 2016.

[10] P. Modica, A. Capocchi, I. Foroni, and M. Zenga, "An assessment of the implementation of the European Tourism Indicator System for sustainable destinations in Italy," *Sustainability*, vol. 10, no. 9, p. 3160, 2018.

[11] M. Fuchs, W. Höpken, and M. Lexhagen, "Big data analytics for knowledge generation in tourism destinations – A case from Sweden," *Journal of Destination Marketing & Management*, vol. 3, no. 4, pp. 198–209, 2014.

[12] J. A. Ivars-Baidal, M. A. Celdrán-Bernabeu, F. Femenia-Serra, J. F. Perles-Ribes, and D. Giner-Sánchez, "Measuring the progress of smart destinations: The use of indicators as a management tool," *Journal of Destination Marketing & Management*, vol. 19, p. 100531, 2021.

[13] J. A. Ivars-Baidal, J. F. Vera-Rebollo, J. Perles-Ribes, F. Femenia-Serra, and M. A. Celdrán-Bernabeu, "Sustainable tourism indicators: what's new within the smart city/destination approach?" *Journal of Sustainable Tourism*, vol. 31, no. 7, pp. 1556–1582, 2023.

[14] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, and K. Tei, "Generative AI for self-adaptive systems: State of the art and research roadmap," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 19, no. 3, pp. 13:1–13:60, 2024.

[15] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[16] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1482–1494. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00129

[17] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[18] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10-16, 2023*, 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html

[19] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=KuPixIqPiq

[20] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: https://openreview.net/forum?id=y0GJXRungR

[21] A. Marginéan, J. Bader, S. Chandra, M. Harman, Y. Jia, K. Mao, A. Mols, and A. Scott, "SapFix: Automated end-to-end repair at scale," in *Proc. 41st Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 269–278.

[22] J. Bader, A. Scott, M. Pradel, and S. Chandra, "Getafix: Learning to fix bugs automatically," *Proc. ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 159:1–159:27, 2019.

[23] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. 47th Int. Conf. on Software Engineering (ICSE)*, 2025, pp. 2188–2200, arXiv:2403.17134.

[24] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language models enable simple systems for generating structured views of heterogeneous data lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023.

[25] W. Huang, Z. Gu, C. Peng, J. Liang, Z. Li, Y. Xiao, L. Wen, and Z. Chen, "AutoScraper: A progressive understanding web agent for web scraper generation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 2371–2389. [Online]. Available: https://aclanthology.org/2024.emnlp-main.141/

[26] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. I. Wang, and T. Yu, "Spider 2.0: Evaluating language models on real-world enterprise Text-to-SQL workflows," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025, arXiv:2411.07763.

[27] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?" *Proceedings of the VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.

[28] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: Instruction-tuning local large language models for data preprocessing," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 8754–8782.

[29] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, S. Han, D. Zhang, and S. Chaudhuri, "Auto-Validate by-History: Auto-program data quality constraints to validate recurring data pipelines," in *Proc. 29th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2023, pp. 4991–5003.

[30] V. Crescenzi, G. Mecca, and P. Merialdo, "RoadRunner: Towards automatic data extraction from large web sites," in *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann, 2001, pp. 109–118.

[31] A. H. F. Laender, B. A. Ribeiro-Neto, A. S. da Silva, and J. S. Teixeira, "A brief survey of web data extraction tools," *SIGMOD Record*, vol. 31, no. 2, pp. 84–93, 2002.

[32] E. Ferrara, P. De Meo, G. Fiumara, and R. Baumgartner, "Web data extraction, applications and techniques: A survey," *Knowledge-Based Systems*, vol. 70, pp. 301–323, 2014.

[33] N. Kushmerick, "Wrapper verification," *World Wide Web*, vol. 3, no. 2, pp. 79–94, 2000.

[34] P. Vassiliadis, A. Simitsis, and S. Skiadopoulos, "Conceptual modeling for ETL processes," in *Proceedings of the 5th ACM International Workshop on Data Warehousing and OLAP (DOLAP '02)*. McLean, VA, USA: ACM, 2002, pp. 14–21.

[35] P. Vassiliadis, "A survey of Extract–Transform–Load technology," *International Journal of Data Warehousing and Mining*, vol. 5, no. 3, pp. 1–27, 2009.

[36] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.

[37] M. Stonebraker, D. Bruckner, I. F. Ilyas, G. Beskales, M. Cherniack, S. B. Zdonik, A. Pagan, and S. Xu, "Data curation at scale: The Data Tamer system," in *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org, 2013. [Online]. Available: http://cidrdb.org/cidr2013/Papers/CIDR13_Paper28.pdf

[38] T. Furche, G. Gottlob, L. Libkin, G. Orsi, and N. W. Paton, "Data wrangling for big data: Challenges and opportunities," in *Proceedings of the 19th International Conference on Extending Database Technology (EDBT 2016)*. Bordeaux, France: OpenProceedings.org, 2016, pp. 473–478.

[39] M. Franklin, A. Halevy, and D. Maier, "From databases to dataspaces: A new abstraction for information management," *ACM SIGMOD Record*, vol. 34, no. 4, pp. 27–33, 2005.

[40] J. Duggan, A. J. Elmore, M. Stonebraker, M. Balazinska, B. Howe, J. Kepner, S. Madden, D. Maier, T. Mattson, and S. B. Zdonik, "The BigDAWG polystore system," *ACM SIGMOD Record*, vol. 44, no. 2, pp. 11–16, 2015.